



DHANALAKSHMI SRINIVASAN UNIVERSITY

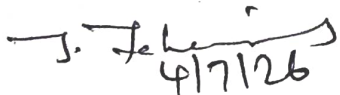
SAMAYAPURAM, TRICHY - 621112

SCHOOL OF ENGINEERING AND TECHNOLOGY

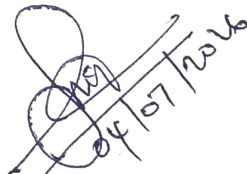
Department of Computer Science and Engineering

PRACTICAL RECORD

24CSE405- OBJECT ORIENTED PROGRAMMING


4/7/26

**Prepared By
J.Fahamitha,
Assistant Professor,
Department of CSE**


04/07/2026

**Approved By,
Dr.J.Mercy Geraldine
Professor & HOD
Department of CSE**



DHANALAKSHMI SRINIVASAN UNIVERSITY

SAMAYAPURAM, TRICHY - 621112

SCHOOL OF ENGINEERING AND TECHNOLOGY

PRACTICAL RECORD

24CSE405- OBJECT ORIENTED PROGRAMMING

Name : _____

Register No. : _____

Semester : _____

Course : _____



DHANALAKSHMI SRINIVASAN UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Name : _____

Register No. : _____

Department : _____

Semester : _____

Course Name : _____

Course Code : _____

Certified that this is a Bonafide Record of the Practical work done for
the above course in Laboratory during the period

Faculty in-charge

Head of the Department

Submitted for the University Practical Examination held on

.....

Internal Examiner

External Examiner

24CSE405 OBJECT ORIENTED PROGRAMMING LAB CONTENTS

EXP NO	TITLE	PAGE NO	MARKS	SIGNATURE
1	Implementation of Classes and objects.			
2	Implementation of Constructors.			
3	Implementation of Method overloading.			
4	Implementation of Inheritance.			
5	Implementation of Interface.			
6	Implementation of Packages.			
7	Implementation of Multithreading.			
8	Implementation of Exception handling			

COURSE OBJECTIVES

- Understand object-oriented programming concepts and fundamentals of Java.
- Understand operators and statements.
- Apply arrays and inheritance concepts.
- Apply packages, threads, and exception handling concepts.
- Implement database connectivity.

LIST OF EXERIMNETS

1. Implementation of Classes and objects.
2. Implementation of Constructors.
3. Implementation of Method overloading.
4. Implementation of Inheritance.
5. Implementation of Interface.
6. Implementation of Packages.
7. Implementation of Multithreading.
8. Implementation of Expection handling

TOTAL 30 PERIODS**COURSE OUTCOMES**

Upon completion, students will be able to:

- CO1: Understand fundamentals of Java.
- CO2: Understand Java operators and decision-making statements.
- CO3: Apply class and inheritance concepts.
- CO4: Apply packages, threads, and exception handling to solve problems.
- CO5: Implement AWT and database connectivity.

1.Implementation of Classes and objects.

AIM:

To implement a Java program demonstrating the use of classes and objects by defining a class Employee and creating objects to access and display employee data.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a class named Employee with the following attributes:

empld (int)

name (String)

salary (double)

Step 3: Create a constructor in the class to initialize these attributes.

Step 4: Define a method displayDetails() inside the class to print employee details.

Step 5: Create another class called EmployeeMain that contains the main() method.

Step 6: Inside the main() method:

Create objects of the Employee class.

Pass values through the constructor while creating the objects.

Call the displayDetails() method using each object.

Step 7: Display the output.

Step 8: End the program.

PROGRAM:

```
// Employee.java
```

```
class Employee {
```

```
    // Attributes
```

```
    int empld;
```

```
    String name;
```

```
    double salary;
```

```
    // Constructor
```

```
    Employee(int id, String n, double s) {
```

```
        empld = id;
```

```

        name = n;

        salary = s;
    }

    // Method to display employee details
    void displayDetails() {
        System.out.println("Employee ID: " + empld);
        System.out.println("Name: " + name);
        System.out.println("Salary: ₹" + salary);
        System.out.println("-----");
    }
}

// Main class with main() method
public class EmployeeMain {
    public static void main(String[] args) {
        // Creating objects of Employee class
        Employee e1 = new Employee(101, "Alice", 50000.0);
        Employee e2 = new Employee(102, "Bob", 60000.0);
        // Displaying employee details
        e1.displayDetails();
        e2.displayDetails();
    }
}

```

OUTPUT:

Employee ID: 101

Name: Alice

Salary: ₹50000.0

Employee ID: 102

Name: Bob

Salary: ₹60000.0

RESULT:

The Java program was successfully executed. The experiment clearly demonstrated how to define a class, create objects, use a constructor, and access data using methods.

2.Implementation of Constructors

AIM:

To write a Java program that demonstrates the use of constructors by creating a class Car and initializing object data using a parameterized constructor.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a class named Car.

Step 3: Declare attributes such as:

String brand

String model

int year

Step 4: Create a parameterized constructor to initialize the attributes when an object is created.

Step 5: Define a method displayDetails() to print car details.

Step 6: Create another class CarMain with the main() method.

Step 7: In the main() method:

Create objects of the Car class by passing values to the constructor.

Call the displayDetails() method to display car information.

Step 8: Compile and run the program.

Step 9: End the program.

PROGRAM:

```
// Car.java
```

```
class Car {
```

```
    // Attributes
```

```
    String brand;
```

```
    String model;
```

```
    int year;
```

```
    // Parameterized Constructor
```

```
    Car(String b, String m, int y) {
```

```
        brand = b;
```

```
        model = m;
```

```
        year = y;
    }

    // Method to display car details
    void displayDetails() {
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
        System.out.println("-----");
    }
}

// Main Class
public class CarMain {
    public static void main(String[] args) {
        // Creating objects using constructor
        Car car1 = new Car("Toyota", "Camry", 2022);
        Car car2 = new Car("Honda", "Civic", 2021);
        // Displaying details
        car1.displayDetails();
        car2.displayDetails();
    }
}
```

OUTPUT:

Brand: Toyota

Model: Camry

Year: 2022

Brand: Honda

Model: Civic

Year: 2021

RESULT:

The Java program was successfully executed. This experiment demonstrated how to define and use a parameterized constructor to initialize object data at the time of creation.

3.Implementation of Method overloading.

AIM:

To implement a Java program that demonstrates method overloading by defining multiple versions of a method with the same name but different parameters.

ALGORITHM:

Step 1: Start the program.

Step 2: Define a class named Calculator.

Step 3: Inside the class, define multiple methods with the same name add() but with different parameter types or counts:

add(int a, int b)

add(double a, double b)

add(int a, int b, int c)

Step 4: Each method should return or print the sum of the parameters.

Step 5: Create another class called CalculatorMain with the main() method.

Step 6: In the main() method:

Create an object of the Calculator class.

Call all versions of the add() method with different arguments.

Step 7: Observe how the correct method version is called based on the arguments.

Step 8: Compile and run the program.

Step 9: End the program.

PROGRAM:

```
// Calculator.java
```

```
class Calculator {
```

```
    // Method 1: Two integers
```

```
    void add(int a, int b) {
```

```
        System.out.println("Sum of two integers: " + (a + b));
```

```
    }
```

```
    // Method 2: Two doubles
```

```
    void add(double a, double b)
```

```
    { System.out.println("Sum of two doubles: " + (a +  
        b));
```

```
}  
  
// Method 3: Three integers  
void add(int a, int b, int c) {  
    System.out.println("Sum of three integers: " + (a + b + c));  
}  
  
}  
  
// Main class  
public class CalculatorMain {  
    public static void main(String[] args) {  
        // Creating object  
        Calculator calc = new Calculator();  
        // Calling overloaded methods  
        calc.add(10, 20);           // Calls method with 2 int parameters  
        calc.add(5.5, 4.5);        // Calls method with 2 double parameters  
        calc.add(1, 2, 3);         // Calls method with 3 int parameters  
    }  
}
```

OUTPUT:

Sum of two integers: 30

Sum of two doubles: 10.0

Sum of three integers: 6

RESULT:

The program executed successfully. The concept of method overloading was demonstrated clearly using methods with the same name but different parameter lists. Java's compile-time polymorphism was observed.

4.Implementation of Inheritance.

Aim:

To implement inheritance in a programming language (Java/Python/C++) and understand its benefits and applications.

Algorithm:

1. Define a base class (Parent/Superclass) with attributes and methods.
2. Define a derived class (Child/Subclass) that inherits from the base class.
3. Add specific attributes and methods to the derived class.
4. Create objects of both classes and demonstrate inheritance.

Program (Java):

// Base class: Vehicle

```
class Vehicle {  
    String brand;  
    int wheels;  
    Vehicle(String brand, int wheels)  
    { this.brand = brand;  
      this.wheels = wheels;  
    }  
    void displayDetails()  
    { System.out.println("Brand: " + brand);  
      System.out.println("Wheels: " + wheels);  
    }  
}
```

// Derived class: Car

```
class Car extends Vehicle  
{ int doors;  
  Car(String brand, int wheels, int doors)  
  { super(brand, wheels);  
    this.doors = doors;  
  }  
  void displayCarDetails() {
```

```

        displayDetails();

        System.out.println("Doors: " + doors);
    }
}

public class Main {

    public static void main(String[] args) {

        Vehicle vehicle = new Vehicle("Generic", 4);

        vehicle.displayDetails();

        Car car = new Car("Toyota", 4, 4);

        car.displayCarDetails();

    }

}

```

Program (Python):

Base class: Vehicle

class Vehicle:

```

    def __init__(self, brand, wheels):

        self.brand = brand

        self.wheels = wheels

    def display_details(self):

        print("Brand:", self.brand)

        print("Wheels:", self.wheels)

```

Derived class: Car

class Car(Vehicle):

```

    def __init__(self, brand, wheels, doors):

        super().__init__(brand, wheels)

        self.doors = doors

    def display_car_details(self):

        self.display_details()

        print("Doors:", self.doors)

```

```

vehicle = Vehicle("Generic", 4)

```

```

vehicle.display_details()

```

```

car = Car("Toyota", 4, 4)

```

```
car.display_car_details()
```

Output:

Java:

Brand: Generic

Wheels: 4

Brand: Toyota

Wheels: 4

Doors: 4

Python:

Brand: Generic

Wheels: 4

Brand: Toyota

Wheels: 4

Doors: 4

Result:

The program demonstrates the implementation of inheritance, where the Car class inherits attributes and methods from the Vehicle class and adds its own specific attributes and methods. The output shows that the Car class can access and use the attributes and methods of the Vehicle class.

5. Implementation of Interface.

AIM:

To write a Java program that demonstrates the concept of interfaces by defining an interface Shape and implementing it in classes Circle and Rectangle.

ALGORITHM:

Step 1: Start the program.

Step 2: Define an interface named Shape with a method calculateArea().

Step 3: Create a class Circle that implements the Shape interface.

Step 4: In the Circle class:

Declare attribute radius.

Implement the method calculateArea() to calculate the area of the circle.

Step 5: Create another class Rectangle that also implements the Shape interface.

Step 6: In the Rectangle class:

Declare attributes length and breadth.

Implement the method calculateArea() to calculate the area of the rectangle.

Step 7: Create a class InterfaceMain that contains the main() method.

Step 8: In the main() method:

Create objects of Circle and Rectangle.

Call the calculateArea() method using each object.

Step 9: Compile and run the program.

Step 10: End the program.

PROGRAM:

```
// Interface
```

```
interface Shape {  
    void calculateArea();  
}
```

```
// Circle class implementing Shape interface
```

```
class Circle implements Shape {
```

```
double radius;

Circle(double radius) {
    this.radius = radius;
}

public void calculateArea() {
    double area = Math.PI * radius * radius;
    System.out.println("Area of Circle: " + area);
}
}

// Rectangle class implementing Shape interface
class Rectangle implements Shape {
    double length, breadth;

    Rectangle(double l, double b) {
        length = l;
        breadth = b;
    }

    public void calculateArea()
    { double area = length * breadth;
      System.out.println("Area of Rectangle: " + area);
    }
}

// Main class
public class InterfaceMain {
    public static void main(String[] args)
    { Shape circle = new Circle(5.0);
      Shape rectangle = new Rectangle(4.0, 6.0);
      circle.calculateArea();
      rectangle.calculateArea();
    }
}
```

OUTPUT:

Area of Circle: 78.53981633974483

Area of Rectangle: 24.0

RESULT:

The Java program was successfully executed. The concept of interfaces was demonstrated by implementing the Shape interface in multiple classes and using polymorphism to call the overridden method.

6.Implementation of Packages.

AIM:

To implement a Java program that demonstrates the concept of packages by creating a user-defined package and accessing its class and methods from another class.

ALGORITHM:

- Step 1: Create a folder named mypack to represent a user-defined package.
- Step 2: Inside the mypack folder, create a Java file named Greet.java.
- Step 3: Define the package using package mypack; at the top of the file.
- Step 4: Create a class Greet with a method sayHello() that prints a greeting.
- Step 5: Compile the file using javac -d . Greet.java to generate the package structure.
- Step 6: Create a new Java file named MainClass.java in the parent directory.
- Step 7: Import the package using import mypack.Greet;.
- Step 8: In MainClass, create an object of Greet and call the sayHello() method.
- Step 9: Compile and run MainClass.java.
- Step 10: End the program.

PROGRAM:

```
import java.util.*;

class AdvancedCalculator
{
    public int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++)
            { fact *= i;
        }
        return fact;
    }

    public boolean isPrime(int n) {
```

```

    if (n <= 1) return false;
    for (int i = 2; i <= Math.sqrt(n); i++)
        { if (n % i == 0)
            return false;
        }
    return true;
}

public boolean isEven(int n)
{ return n % 2 == 0;
}
}

// This is the entry point
class Main {
    public static void main(String[] args)
    { AdvancedCalculator calc = new
      AdvancedCalculator(); int a = 5, b = 10;

      System.out.println("Factorial of " + a + ": " + calc.factorial(a));
      System.out.println("Is " + a + " prime? " + calc.isPrime(a));
      System.out.println("Is " + b + " even? " + calc.isEven(b));
    }
}

```

OUTPUT:

Factorial of 5: 120

Is 5 prime? true

Is 10 even? true

RESULT:

The program successfully demonstrated the use of user-defined packages in Java. The class Greet was accessed from the package mypack using the import keyword.

7.Implementation of Multithreading.

AIM:

To implement a Java program that demonstrates multithreading using the Thread class to run multiple tasks concurrently.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a class MyThread that extends the Thread class.

Step 3: Override the run() method to define the task the thread will execute.

Step 4: In the main() method, create multiple objects of MyThread.

Step 5: Use the start() method to begin execution of each thread.

Step 6: Observe the interleaved output showing concurrent thread execution.

Step 7: End the program.

PROGRAM:

// Class extending Thread

class MyThread extends Thread

{ public void run() {

for (int i = 1; i <= 5; i++)

{ System.out.println(Thread.currentThread().getName() + " - Count: " + i);

try {

Thread.sleep(500); // Pause for 500ms

} catch (InterruptedException e)

{ System.out.println(e);

}

}

}

}

// Main class

public class MultiThreadDemo {

public static void main(String[] args) {

```
MyThread t1 = new MyThread();  
    MyThread t2 = new MyThread();  
    t1.setName("Thread A");  
    t2.setName("Thread B");  
    t1.start();  
    t2.start();  
}  
}
```

OUTPUT:

Thread A - Count: 1
Thread B - Count: 1
Thread A - Count: 2
Thread B - Count: 2
Thread A - Count: 3
Thread B - Count: 3
Thread A - Count: 4
Thread B - Count: 4
Thread A - Count: 5
Thread B - Count: 5

RESULT:

The program executed successfully and demonstrated the concept of multithreading, where multiple threads run concurrently using the Thread class.

8.Implementation of Expection handling

AIM:

To implement a Java program that demonstrates exception handling using try, catch, and finally blocks.

ALGORITHM:

Step 1: Start the program.

Step 2: Declare two integer variables, one as numerator and another as denominator.

Step 3: Use a try block to perform division of numerator by denominator.

Step 4: Catch the exception using a catch block (e.g., ArithmeticException).

Step 5: Display an appropriate error message inside the catch block.

Step 6: Use a finally block to print a message that always executes.

Step 7: End the program.

PROGRAM:

```
public class ExceptionHandlingDemo
{
    public static void main(String[] args) {
        int numerator = 10;
        int denominator = 0;

        try {
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        } finally {
            System.out.println("This block always executes.");
        }
    }
}
```

OUTPUT:

Error: Division by zero is not allowed.

This block always executes.

RESULT:

The program executed successfully and demonstrated how exceptions are handled in Java using try, catch, and finally blocks.

